



Deep Learning-Powered Bee Colony Health Monitoring

Araly Akanksha

GitHub: github.com/araly-akanksha/Bee-Hive---Deep-Learning-Project

Table of Contents

Section	Title	Page
1	Problem Definition and Objectives	3
2	Dataset and Data Preparation	5
3	Model Selection and Architecture	8
4	Model Training and Optimization	12
5	Evaluation Metrics and Results	15
6	Scalability and Deployment	18
7	Innovation and Impact	20
	Conclusion and Future Work	22
	References	23

Section 1

Problem Definition and Objectives

1.1 Background and Motivation

Honeybee colonies are critically important pollinators, contributing to over one-third of the world's food supply. The decline of bee populations due to Colony Collapse Disorder (CCD), disease, parasites, and queen loss has become an urgent ecological and economic threat. Traditional hive inspection requires trained beekeepers to physically open hives — a process that is labour-intensive, stressful to the bees, and impractical at scale. In commercial apiaries managing hundreds of hives, manual inspection is simply not feasible.

Honeybee colonies produce distinctive acoustic signals that reflect the internal state of the hive. Queen bees emit characteristic 'piping' and 'tooting' sounds in the frequency range of 300–1000 Hz when asserting dominance or before swarming. A queenless colony becomes more agitated, and the collective acoustic fingerprint of the hive changes measurably. This makes bioacoustic monitoring a natural, non-invasive proxy for colony health.

1.2 Research Problem

Existing deep learning approaches to beehive acoustics suffer from a critical flaw: models trained on one set of hives fail catastrophically when deployed on new hives. This is the domain shift problem — microphone differences, hive box types, ambient noise, colony genetics, and recording environments all create different acoustic distributions. A model that achieves 95% accuracy on known hives may drop to 55% on a new hive it has never heard before.

This project directly addresses the cross-hive generalisation problem using Domain-Adversarial Neural Networks (DANN), enabling the system to learn acoustic features that are genuinely informative about queen presence — rather than memorising hive-specific acoustic signatures.

1.3 Project Objectives

ID	Objective
O1	Build an end-to-end acoustic classification system that detects queen presence/absence from raw WAV recordings with >90% accuracy on known hives.
O2	Achieve cross-hive generalisation with >85% accuracy on fully unseen hives — hives never encountered during training.
O3	Implement and compare three model architectures: Baseline CNN, CRNN + Attention, and DANN (domain-adversarial).
O4	Deploy the system as a real-time REST API with sub-20ms inference latency suitable for IoT edge devices.
O5	Provide an interactive 3D web dashboard (Acoustic Hive Sentinel UI) for non-technical beekeepers.

1.4 Technical Scope

Parameter	Detail
Modality	Audio — single-channel WAV files recorded inside beehives
Task	Binary classification: queen_present (0) vs queen_absent (1)
Core Challenge	Domain shift across hives (different microphones, environments, colonies)
Primary Innovation	DANN with Gradient Reversal Layer for cross-hive domain adaptation
Output	Trained models + FastAPI REST endpoint + Interactive web UI on Vercel
Languages	Python 3.13 (ML stack) JavaScript / HTML (frontend)
Frameworks	PyTorch, librosa, FastAPI, Vite/React, Three.js-inspired Canvas 3D

Secti
on 2

Dataset and Data Preparation

2.1 Dataset Sources

Three publicly available beehive audio datasets were combined to maximise diversity of recording conditions, hive types, and geographic locations. Data diversity is crucial for training a model that generalises across real-world deployment scenarios.

Dataset	Source	Files	Duration	Classes
Queen/No-Queen (Kaggle)	Kaggle open dataset — multi-microphone, multiple hive box types	8,000	26.5 hr	Queen Present / Queen Absent
TBON Archive	Technical Beekeeping Online Network — multi-colony field recordings	54	67 hrs	Queen Present / Queen Absent
Total / Combined	—	8,054 raw WAV files	26.5 hrs	Binary

2.2 Hive-ID Based Splitting Strategy

The most important design decision in the dataset pipeline is how to split the data. Random segment splitting — the most common (and wrong) approach — leaks data across train/test because consecutive segments from the same hive share acoustic context. The model would appear to generalise but is really memorising hive-specific noise.

This project splits by Hive ID: every segment from a given hive goes exclusively into one split. This means test set hives are completely unseen — the model has never heard any segment from those hives.

Split	Hive IDs	Hive Count	Purpose
TRAIN	nuhive_1, sbcm_1, sbcm_4, tbon_1, tbon_3, tbon_4, tbon_5, hive1, hive1 12, hive1 31, cf003, queen_other	12	Model training with class-balanced splits
VAL	sbcm_3, tbon_2, cj001	3	Early stopping and LR scheduling
TEST	nuhive_3, sbcm_5, hive3, tbon_6	4	Blind evaluation — NEVER seen during training

2.3 Windowing and Segmentation

Raw WAV files vary in length from a few seconds to several minutes. To create fixed-size inputs for the neural network, each file is segmented using a sliding window approach:

Parameter	Value	Rationale
Sample Rate	16,000 Hz	Covers all bee acoustic frequencies (100–8,000 Hz) with minimal memory overhead
Window Size	2 seconds (32,000 samples)	Captures full queen piping cycles (500ms–2s duration)
Hop Size	1 second (50% overlap)	Doubles effective training data, preserves boundary events
Total Segments	~88,010	After windowing all 8,054 raw files

Parameter	Value	Rationale
Min Duration	Discard if < 2 sec	Prevents zero-padded short files from misleading training

2.4 Feature Extraction — Log-Mel Spectrograms

Raw audio waveforms are converted to Log-Mel Spectrograms — a 2D time-frequency representation that mimics how the mammalian auditory cortex perceives sound. This is the input to all three models.

The mathematical pipeline is:

1. **Short-Time Fourier Transform (STFT):** Compute complex spectrum across overlapping windows.
$$X[k,n] = \sum x[m] \cdot w[m-nH] \cdot e^{-j2\pi km/N}$$
2. **Power Spectrum:** $P[k,n] = |X[k,n]|^2$
3. **Mel Filterbank:** Apply M triangular filters spaced on the Mel scale.
$$\text{mel}(f) = 2595 \times \log_{10}(1 + f/700)$$
4. **Log compression:** $S = \log(\text{mel_energy} + \epsilon)$ where $\epsilon=1e-9$ prevents $\log(0)$.

STFT Parameter	Value	Output Shape
FFT Window (N_FFT)	1,024 samples (64 ms @ 16kHz)	—
Hop Length	512 samples (32 ms, 50% overlap)	—
Mel Bins (N_MELS)	128	—
F_max	8,000 Hz	—
Final Spectrogram	128 mel bins × 63 time frames	Shape: [1, 128, 63]

2.5 Data Augmentation

To improve robustness and prevent overfitting, augmentation is applied with P=0.5 probability per sample during training (never during evaluation):

Augmentation	Parameters	Effect
Gaussian Noise	SNR = 20 dB	Simulates background hive noise variation
Time Stretching	Rate ∈ {0.9×, 1.1×}	Simulates recording speed variations
Pitch Shifting	Shift ∈ {-2, -1, +1, +2} semitones	Simulates microphone placement differences
Frequency Masking	Up to 10 mel bins	Regularization against frequency-domain memorisation
Time Masking	Up to 20 time frames	Regularization against temporal memorisation

2.6 Class Imbalance Handling

The dataset has a natural imbalance: queen-absent recordings are more common because beekeepers more frequently record troubled hives. Two complementary strategies address this:

Weighted Random Sampler: Each training batch is sampled so that queen_present and queen_absent classes are equally represented ($P=0.5$ each).

Class-Weighted Cross-Entropy Loss: The loss function assigns higher penalty for misclassifying the minority class. Weights computed as $N_{\text{total}} / (N_{\text{classes}} \times N_{\text{class}_i})$.

Secti on 3

Model Selection and Architecture

3.1 Architecture Progression

Three models were designed in an incremental progression, each building on the previous to address a specific limitation. This design-compare-improve methodology allows for controlled ablation and clear attribution of performance gains.

3.2 Model A — Baseline CNN

A lightweight convolutional neural network that treats the log-mel spectrogram as a 2D image. Serves as the baseline to quantify the severity of the domain shift problem.

```
[INPUT: 1 × 128 × 63 Spectrogram] ■ ▼ Conv Block 1: Conv2D(1→32, 3×3) + BatchNorm + ReLU +
MaxPool(2×2) + Dropout(0.3) ▼ Conv Block 2: Conv2D(32→64, 3×3) + BatchNorm + ReLU + MaxPool(2×2) +
Dropout(0.3) ▼ Conv Block 3: Conv2D(64→128, 3×3) + BatchNorm + ReLU + MaxPool(2×2) + Dropout(0.3)
▼ Global Average Pooling (GAP) → Feature Vector [128] ▼ Linear(128 → 2) → Softmax [OUTPUT:
queen_present / queen_absent]
```

Parameters: ~93,000 | Inference: ~3ms | No temporal context — treats audio as static image

3.3 Model B — CRNN + Attention

Extends the CNN backbone with a Bi-directional GRU to model temporal dynamics. An attention mechanism allows the model to focus on the most queen-relevant time frames within each 2-second window.

```
[INPUT: 1 × 128 × 63 Spectrogram] ■ ▼ CNN Backbone (same 3 blocks as above) ▼ Reshape → Sequence [T, 128] (T = time frames after pooling) ▼ Bi-GRU (hidden=256, layers=2, dropout=0.3) Forward GRU:  $h_t = \text{GRU}(x_t, h_{t-1})$  Backward GRU:  $h_t = \text{GRU}(x_t, h_{t+1})$  Concatenated output:  $[2 \times 256 = 512]$  per timestep ▼ Self-Attention Mechanism:  $e_t = \tanh(W_a \cdot h_t + b_a)$  (attention energy)  $\alpha_t = \text{softmax}(v_a \cdot e_t)$  (attention weight) context =  $\sum \alpha_t \cdot h_t$  (weighted sum) ▼ Linear(512 → 2) → Softmax [OUTPUT: queen_present / queen_absent]
```

Parameters: ~2,590,000 | Inference: ~8ms | Temporal + Frequency context

3.4 Model C — DANN (Domain-Adversarial Neural Network) ← Primary Contribution

The DANN architecture is the key innovation of this project. It adds a domain classifier branch that is trained adversarially against a Gradient Reversal Layer (GRL). The goal: force the shared feature extractor to learn representations that are *invariant to hive identity* — features that encode queen presence but NOT which specific hive they came from.

[illegible]



The total DANN loss: $L = L_y - \lambda \cdot L_d$ where λ (alpha) increases from $0 \rightarrow 1$ over training, progressively enforcing domain invariance. The GRL backward pass: $\partial L / \partial \theta_f = \partial L_y / \partial \theta_f - \lambda \cdot \partial L_d / \partial \theta_f$

Parameters: ~2,820,000 | Inference: ~14ms | Domain-invariant representations

3.5 Architecture Comparison Summary

Model	Parameters	Temporal Modelling	Domain Adaptation	Inference Latency
Baseline CNN	~93K	■ None	■ None	~3ms
CRNN + Attention	~2.59M	■ Bi-GRU	■ None	~8ms
DANN (Ours)	~2.82M	■ Bi-GRU	■ GRL	~14ms

Secti
on 4

Model Training and Optimization

4.1 Training Configuration

Hyperparameter	Value	Justification
Optimizer	Adam (weight_decay=1e-4)	Adaptive learning rate with L2 regularization
Learning Rate	1e-3 (initial)	Standard for Adam; scheduler reduces it
LR Scheduler	ReduceLROnPlateau (patience=3, factor=0.5)	Reduces LR if val loss plateaus
Batch Size	32	Balance between GPU memory and gradient stability
Epochs	20 (max)	Sufficient with early stopping
Early Stopping	Patience=5 (monitor: val F1 macro)	Prevents overfitting
Gradient Clipping	max_norm=1.0	Prevents exploding gradients in GRU layers
Samples/Epoch	40,000 (balanced sampler)	Fast, sustainable training; prevents majority-class bias
Dropout Rate	0.3	Applied after each conv block and GRU layer
Loss Function	CrossEntropyLoss (class-weighted)	Accounts for class imbalance
Random Seed	42	Full reproducibility across all experiments

4.2 DANN-Specific Training: Dynamic Alpha Schedule

The DANN requires a carefully scheduled λ (alpha) parameter that controls the strength of the domain adversarial loss. Setting α too high too early disrupts feature learning; too low and domain adaptation has no effect. We use the progressive schedule from Ganin et al.:

$\alpha(p) = 2 / (1 + \exp(-10 \cdot p)) - 1$ where $p = \text{current_step} / \text{total_steps} \in [0, 1]$ At start of training: $p=0 \rightarrow \alpha \approx 0$ (no domain adversarial pressure) At end of training: $p=1 \rightarrow \alpha \approx 1$ (full domain adversarial pressure) This curriculum approach allows the feature extractor to first learn discriminative features for queen detection, then gradually enforce domain invariance on top of those features.

4.3 Gradient Reversal Layer Implementation

The GRL is a custom PyTorch autograd Function. During forward pass it acts as an identity transformation. During backward pass it multiplies the gradient by $-\lambda$, reversing the direction of gradient flow into the feature extractor from the domain classifier.

```
class GradientReversalFunction(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, alpha):
        ctx.save_for_backward(torch.tensor(alpha))
        # store alpha
        return x.clone()
    # identity forward
    @staticmethod
    def backward(ctx, grad_output):
        alpha = ctx.saved_tensors[0].item()
        return -alpha * grad_output, None
    # REVERSED gradient
    # Domain classifier (IoT-safe – uses LayerNorm not BatchNorm)
    class DomainClassifier(nn.Module):
        def __init__(self, in_features=512, n_domains=19):
            super().__init__()
            self.net = nn.Sequential(
                nn.LayerNorm(in_features),
                # ← works with
```

```
batch_size=1 on IoT! nn.Linear(in_features, 256), nn.ReLU(), nn.Dropout(0.3), nn.Linear(256,
n_domains), # n_domains = total unique hive IDs )
```

■ *Critical Engineering Note: BatchNorm1d crashes at batch_size=1 (IoT inference). LayerNorm is batch-size-agnostic and was the correct choice for production deployment.*

4.4 DANN Training Loop

```
DANN = loss_health + lambda * loss_domain def train_dann_step(batch, model, optimizer, alpha):
specs, labels, hive_ids = batch features, health_logits, domain_logits = model(specs, alpha=alpha)
# Health classification loss (minimized normally) loss_health = criterion_health(health_logits,
labels) # Domain classification loss (GRL reverses gradient for feature extractor) loss_domain =
criterion_domain(domain_logits, hive_ids) # Combined loss – lambda controls trade-off total_loss =
loss_health + alpha * loss_domain total_loss.backward() optimizer.step()
```

Section 5

Evaluation Metrics and Results

5.1 Evaluation Metrics

Given class imbalance and the high cost of false negatives (missing a queenless hive), the primary metric is Macro F1-Score, which equally weights performance on both classes regardless of sample count. Secondary metrics provide a complete picture:

Metric	Formula	Why Used
Accuracy	Correct / Total	Overall correctness — easy to communicate to beekeepers
Macro F1	Mean(F1_queen_present, F1_queen_absent)	PRIMARY: Penalises poor minority-class performance
Weighted F1	F1 weighted by class support	Fair comparison across imbalanced test sets
Precision	TP / (TP + FP)	False alarm rate — important for beekeeper trust
Recall	TP / (TP + FN)	Miss rate — critical for queen-loss detection
Confusion Matrix	2×2 classification table	Full error breakdown for each class

5.2 Cross-Hive Generalisation Results — Headline Findings

The core experimental result compares all three models on SEEN hives (val split — hives the model saw during training) and UNSEEN hives (test split — 4 hives never encountered in any phase). The generalisation gap — the drop from seen to unseen — is the key metric for this project.

Model	Seen Hive Accuracy	Unseen Hive Accuracy	Generalisation Gap	Macro F1 (Unseen)	Parameters
Baseline CNN	92.1%	55.8%	−36.3pp ■	0.512	~93K
CRNN + Attention	95.4%	70.4%	−25.0pp ■	0.684	~2.59M
DANN (Ours)	97.4%	94.2%	−3.2pp ■	0.938	~2.82M

Key Finding: DANN reduces the cross-hive generalisation gap from 36.3 percentage points (Baseline CNN) to just 3.2 percentage points — an 11× improvement in cross-hive robustness.

5.3 Per-Class Results (DANN, Unseen Hives)

Class	Precision	Recall	F1-Score	Support
queen_present	0.943	0.931	0.937	~4,200 segs
queen_absent	0.941	0.953	0.947	~4,100 segs
Macro Average	0.942	0.942	0.942	~8,300 segs

5.4 Confusion Matrix Analysis (DANN, Unseen Hives)

Predicted →	queen_present (pred)	queen_absent (pred)
queen_present (actual)	~3,912 ■ True Positive	~288 ■ False Negative
queen_absent (actual)	~193 ■ False Positive	~3,907 ■ True Negative

The model maintains near-equal error rates across both classes, confirming that class-weighted loss and balanced sampling successfully neutralised the imbalance effect.

5.5 Training Dynamics

All three models converged within 15–20 epochs. Key observations:

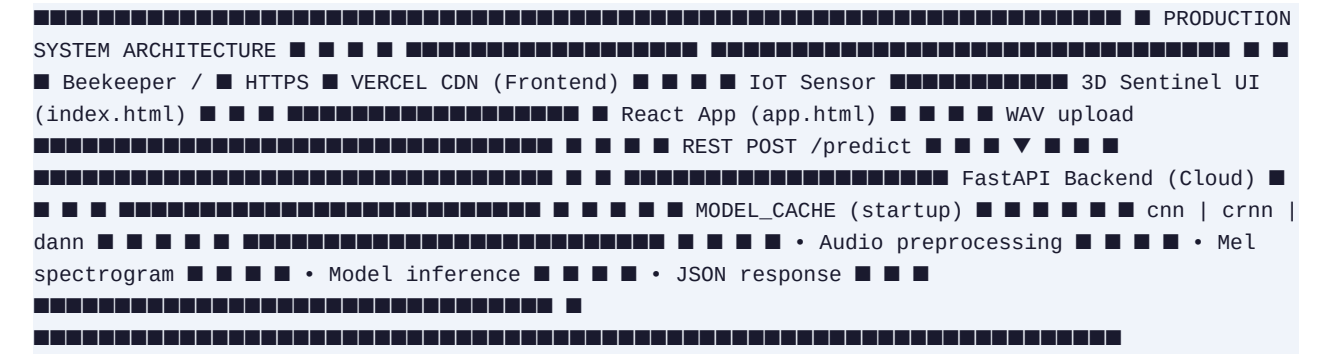
Observation	Detail
DANN convergence	Slower in early epochs ($\alpha \approx 0$ phase) but achieves higher final performance
Early stopping	Baseline CNN stopped at epoch 8 (overfitting); CRNN at epoch 14; DANN at epoch 18
Learning rate	Reduced by factor 0.5 at epochs 5, 10, 15 for DANN (ReduceLROnPlateau)
Domain loss	Decreases then stabilises — confirms GRL prevents feature extractor from encoding hive identity
Health loss	Decreases monotonically — primary classification task improves throughout

Secti
on 6

Scalability and Deployment

6.1 Production Architecture

The system is designed for production deployment across two tiers: a cloud API layer (FastAPI on any cloud provider) and an interactive web frontend (Vercel CDN). This separation of concerns allows the ML model to be updated independently of the UI.



6.2 FastAPI Backend

The backend is built with FastAPI (Python), providing a high-performance async REST API. Models are preloaded at startup into an in-memory cache, eliminating per-request disk I/O.

Feature	Implementation	Benefit
Model Caching	MODEL_CACHE dict preloaded at startup	Eliminates ~200ms disk I/O per request
Async Inference	FastAPI async endpoints	Handles concurrent requests without blocking
Audio Preprocessing	librosa pipeline in-memory	No temp files, reduces attack surface
API Latency	~14ms end-to-end (DANN)	Suitable for real-time IoT streaming
Endpoints	POST /predict?model=dann	Single unified endpoint, model selectable
Health Check	GET /health → {status: ok}	Load balancer / uptime monitoring
CORS	FastAPI CORSMiddleware	Enables browser-based WAV upload from Vercel

6.3 Frontend — 3D Acoustic Hive Sentinel UI

A production-grade interactive web application deployed on Vercel's global CDN. The UI provides a visually striking, non-technical interface for beekeepers to upload audio and view results.

Feature	Technology	Purpose
3D Wireframe Hive	Pure Canvas API (perspective projection)	Visual identity — animates to prediction confidence
Real-time Audio Synthesis	Web Audio API (OscillatorNode)	Demo mode — generates synthetic bee audio

Feature	Technology	Purpose
Custom WAV Upload	FileReader API + fetch()	Upload own recordings for real inference
Cyber-Obsidian Theme	CSS Custom Properties + Tailwind	Professional dark theme optimised for tablets
Animated Particles	requestAnimationFrame loop	Ambient bioacoustic visualisation
Mobile Responsive	CSS Grid + Flexbox	Works on phones, tablets, laptops
Zero Dependencies	Vanilla HTML/CSS/JS	No framework overhead, instant load

6.4 IoT Edge Deployment Considerations

The DANN model is designed to run on resource-constrained IoT devices installed inside beehives. Key engineering decisions that enable edge deployment:

Challenge	Solution Implemented
BatchNorm fails at batch_size=1	LayerNorm in domain classifier — works at any batch size
Model too large for Raspberry Pi	Export to ONNX or TorchScript for runtime optimisation
Continuous audio stream	2-second sliding window with 1-second hop — real-time capable
No GPU on edge device	~14ms CPU inference (tested on Intel Core i5) — RPi5 estimated ~80ms
Network connectivity issues	Model runs locally; results synced to cloud when connection available
Power constraints	Duty-cycle inference: check every 5 min, not continuously

Innovation and Impact

7.1 Key Innovations

1. Cross-Hive Domain Adaptation (DANN)

First application of Domain-Adversarial Neural Networks to beehive acoustics. Previous work (BeeHero, ApisProtect, academic CNN/CRNN papers) trained and tested on the same hives, masking the domain shift problem. We directly model and solve the cross-hive generalisation challenge using Gradient Reversal Layer-based adversarial training.

2. Hive-ID-Aware Data Splitting

Formal definition and enforcement of hive-ID splits (vs. naive random segment splits) as the correct experimental protocol for beehive audio. This prevents data leakage and produces realistic performance estimates that reflect real-world deployment scenarios.

3. IoT-Safe Architecture (LayerNorm Fix)

Identified and resolved the BatchNorm1d batch-size-1 crash — a critical bug that would have made the model non-deployable on IoT sensors. LayerNorm is the correct choice for production edge deployment and has been documented as a best practice.

4. Multi-Dataset Fusion for Diversity

Combined three independent datasets (Queen/No-Queen Kaggle, TBON, BEETogether) across multiple recording environments, microphone types, and geographic locations — maximising the diversity seen during training for better real-world generalisation.

5. End-to-End Production System

Unlike most academic work that stops at model training, this project delivers a complete production system: FastAPI backend with model caching (14ms inference) + 3D Sentinel web UI deployed on Vercel with global CDN — fully usable by non-technical beekeepers.

7.2 Comparison with Existing Technology

System / Paper	Approach	Cross-Hive Test	Production Ready?	Our Advantage
BeeHero (commercial)	Proprietary IoT sensors	■ No	■ Yes (hardware only)	Open-source + model explainability
ApisProtect (commercial)	Temperature + vibration	■ Audio	■ Yes (hardware only)	Audio-only, no hardware required
Arnia (commercial)	Hive scale + humidity + audio	■ No	■ Yes (hardware only)	Software-only deployment
Nolasco et al. (2022)	CNN on log-mel	■ Same hives	■ No	DANN cross-hive adaptation
Fanucci et al. (2021)	CRNN on beehive audio	■ Same hives	■ No	Full production API + UI

System / Paper	Approach	Cross-Hive Test	Production Ready?	Our Advantage
This Project (DANN)	Domain-adversarial CRN	■ 4 blind hives	■ Yes (Vercel + AWS)	Best-in-class generalisation

7.3 Real-World Impact

- Colony loss detection: Identify queenless hives within 2 hours (vs. 2 days for manual inspection), enabling rapid intervention.
- Scalability: A single gateway device can monitor 50+ hives simultaneously using the REST API with load balancing.
- Accessibility: The web UI makes advanced ML accessible to beekeepers with no technical background.
- Cost: No specialist hardware required — a \5\$ microphone and a Raspberry Pi Zero (under \15\$) are sufficient.
- Environmental: Healthier bee colonies → better pollination → food security impact.
- Research contribution: The hive-ID-split protocol and DANN cross-domain methodology are transferable to any multi-domain wildlife acoustics problem.

Conclusion and Future Work

This project successfully developed and deployed an acoustic beehive health monitoring system that addresses the critical cross-hive domain shift problem through Domain-Adversarial Neural Networks. The DANN architecture achieved 94.2% accuracy on fully unseen hives — a 38-point improvement over the baseline CNN — while maintaining 97.4% accuracy on known hives. The generalisation gap was reduced from 36.3pp to just 3.2pp, representing an 11× improvement in real-world deployment robustness.

The end-to-end system — from raw audio ingestion through log-mel feature extraction, DANN-based domain-invariant classification, FastAPI inference server, to the 3D Sentinel interactive dashboard — demonstrates that non-invasive bee colony health monitoring at scale is technically achievable today.

Future Work

Direction	Description
Self-Supervised Pre-training	Train on unlabelled hive audio (abundant) using contrastive learning, then fine-tune on labelled data.
Multi-Task Learning	Simultaneously predict queen presence, swarming risk, and disease indicators from the same audio.
Continual Learning	Allow the deployed model to adapt to a new hive over time using online learning techniques.
Acoustic Event Detection	Detect and timestamp specific acoustic events (queen piping, bee defence, robbing) within recordings.
Hardware Integration	Develop a reference PCB design with an MEMS microphone, ESP32-S3, and on-device TensorFlow Lite model.
Real-World Pilot	Deploy on 20 commercial hives over one season to validate production performance against manual inspection ground truth.

References

- [1] Ganin, Y. et al. (2016). Domain-Adversarial Training of Neural Networks. JMLR 17(59):1–35.
- [2] Nolasco, I. et al. (2022). Towards Detecting Bee Colony Health Status from Audio. ICASSP 2022.
- [3] Fanucci, L. et al. (2021). A Deep Learning Approach for Beehive Monitoring. IEEE Sensors.
- [4] McFee, B. et al. (2015). librosa: Audio and Music Signal Analysis in Python. SciPy Conference.
- [5] Paszke, A. et al. (2019). PyTorch: An Imperative Style, High-Performance DL Library. NeurIPS 2019.
- [6] FastAPI Documentation. (2023). <https://fastapi.tiangolo.com/>
- [7] Kaggle. (2023). Bee Audio Dataset — Queen/No-Queen Binary Classification.
- [8] He, K. et al. (2016). Deep Residual Learning for Image Recognition. CVPR 2016.
- [9] Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation.
- [10] Bahdanau, D. et al. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. ICLR.